Instituto Tecnológico Superior De Rio verde

Materia: Programación orientada a objetos

Profesor: ING. José de Jesús Collazo Reyes

Alumno: Luis Alejandro Domínguez Ramírez

Grupo: 2-A

Nombre de la práctica: Herencia en C++

Fecha: 22/04/2026

INDICE

Contenido

INTRODUCCION 3
PLANTEAMIENTO DEL PROBLEMA 4
DISEÑO DE LA SOLUCION 4
DESARROLLO 5
CAPTURAS DEL CODIGO 6
Resultado en consola: 9
ANALISIS DEL PROGRAMA 9
CONCLUSION 10

INTRODUCCION

En esta práctica se trabaja el tema de la programación orientada a objetos, específicamente el uso de la herencia en C++. La herencia es un concepto muy importante dentro de la programación, ya que permite crear nuevas clases a partir de otras ya existentes, reutilizando código y haciendo los programas más organizados y fáciles de entender. Gracias a esto, no es necesario volver a escribir todo desde cero, sino que se pueden aprovechar características que ya fueron definidas anteriormente.

En este trabajo se desarrolló un programa que simula un sistema de empleados, donde se aplican diferentes tipos de herencia. Se creó una clase base llamada Empleado, la cual contiene datos generales como el nombre, la edad y el salario. A partir de esta clase se derivan otras, como los empleados de tiempo completo y los empleados por horas, cada uno con su propia forma de calcular el salario. Esto permite ver de manera clara cómo una misma estructura puede adaptarse a diferentes situaciones.

Además, también se aplicó la herencia múltiple, que es cuando una clase puede heredar características de más de una clase al mismo tiempo. En este caso se utilizó una clase de prestaciones, que agrega información como el seguro médico, y se combinó con la clase de empleado de tiempo completo. Esto ayuda a entender cómo se pueden integrar varias funcionalidades en una sola clase.

El objetivo de esta práctica es comprender mejor cómo funciona la herencia y cómo se puede aplicar en la vida real al momento de programar. También permite desarrollar habilidades para organizar mejor el código, hacerlo más reutilizable y más fácil de mantener. En general, esta actividad ayuda a reforzar los conocimientos de programación orientada a objetos y su importancia en el desarrollo de software.

PLANTEAMIENTO DEL PROBLEMA

En esta práctica se plantea el desarrollo de un sistema que permita representar diferentes tipos de empleados utilizando programación orientada a objetos en C++. El problema consiste en diseñar una estructura de clases que permita manejar información básica de los empleados, como su nombre, edad y salario, y al mismo tiempo diferenciar la forma en que se calcula el salario dependiendo del tipo de empleado.

Se requiere crear una clase base llamada Empleado, de la cual se derivan otras clases como empleado de tiempo completo y empleado por horas, cada una con sus propias características. Además, se solicita implementar herencia múltiple mediante una clase de prestaciones, que permita agregar beneficios adicionales a ciertos empleados.

El objetivo es modelar correctamente este sistema utilizando herencia, de manera que el programa sea capaz de mostrar la información de cada empleado y calcular su salario según corresponda. De esta forma, se busca aplicar los conceptos vistos en clase y comprender mejor su uso en problemas reales.

DISEÑO DE LA SOLUCION

Para resolver el problema planteado, se diseñó una estructura basada en programación orientada a objetos utilizando clases y herencia. Primero, se creó una clase base llamada Empleado, la cual contiene los atributos generales que todos los empleados comparten, como el nombre, la edad y el salario. Además, esta clase incluye métodos para mostrar la información y calcular el salario.

A partir de esta clase base, se derivaron otras clases para representar distintos tipos de empleados. La primera es la clase TiempoCompleto, que hereda de Empleado y agrega un bono adicional al salario. La segunda es la clase PorHoras, que también hereda de Empleado, pero en este caso el salario se calcula en función de las horas trabajadas y el pago por hora.

También se implementó una clase llamada Prestaciones, la cual contiene información adicional como el seguro médico. Esta clase no depende directamente de Empleado, pero se utiliza para complementar la información de ciertos trabajadores.

Finalmente, se creó una clase llamada CompletoPrest, que utiliza herencia múltiple, ya que hereda tanto de TiempoCompleto como de Prestaciones. Esto permite representar a un empleado de tiempo completo que además cuenta con prestaciones adicionales.

Con este diseño, se logra una estructura clara y organizada, donde cada clase tiene una función específica. Además, el uso de la herencia permite reutilizar código y facilita la extensión del programa en caso de que se quieran agregar más tipos de empleados en el futuro.

DESARROLLO

En esta sección se presenta el código desarrollado en C++ junto con la explicación de sus partes más importantes, con el objetivo de comprender cómo se aplica la herencia en la programación orientada a objetos.

Primero, se utilizó un constructor en cada clase. El constructor es un método especial que se encarga de inicializar los valores de los atributos cuando se crea un objeto. Por ejemplo, en la clase Empleado, el constructor recibe el nombre, la edad y el salario, y los asigna a las variables correspondientes. En las clases derivadas, como TiempoCompleto y PorHoras, también se utilizan constructores que llaman al constructor de la clase base para reutilizar el código.

En cuanto a la herencia, se utilizó la palabra clave “public” para indicar que una clase hereda de otra. Por ejemplo, la clase TiempoCompleto hereda de Empleado usando “: public Empleado”. Esto significa que puede utilizar los atributos y métodos de la clase base. De igual manera, la clase CompletoPrest utiliza herencia múltiple, ya que hereda de TiempoCompleto y de Prestaciones al mismo tiempo.

También se hizo uso de la palabra “override”, la cual permite modificar el comportamiento de un método heredado. En este caso, el método calcular() se redefine en las clases hijas para que cada tipo de empleado tenga una forma diferente de calcular su salario. Por ejemplo, en TiempoCompleto se suma un bono, mientras que en PorHoras se multiplican las horas trabajadas por el pago por hora.

Respecto a los métodos, se implementaron funciones como mostrar() y calcular(), que permiten imprimir la información del empleado y calcular su salario. Estos métodos son reutilizados y modificados según la clase, lo que demuestra el uso del polimorfismo.

Finalmente, en la función main() se crearon objetos de cada clase para probar el funcionamiento del programa. Se instanciaron empleados normales, de tiempo completo, por horas y con prestaciones. Después, se llamaron a sus métodos para mostrar la información y calcular el salario, comprobando que cada clase funciona correctamente según su diseño.

De esta manera, el programa cumple con los objetivos de aplicar herencia, reutilizar código y organizar mejor la información mediante el uso de clases.

CAPTURAS DEL CODIGO

Empleado.h

```
Empleado.h > ...
#ifndef EMPLEADO_H
#define EMPLEADO_H

#include <iostream>
using namespace std;


class Empleado{
protected:
    string nombre;
    int edad;
    double salario;
public:
    Empleado(string, int, double);
    virtual double calcular();
    virtual void mostrar();
};


class TiempoCompleto : public Empleado{
    double bono;
public:
    TiempoCompleto(string, int, double, double);
    double calcular() override;
};

class PorHoras : public Empleado{
    int horas; double pago;
public:
    PorHoras(string, int, int, double);
    double calcular() override;
};


class Prestaciones{
protected:
    string seguro;
```

En esta imagen se muestra el archivo de encabezado (Empleado.h), donde se definen las clases utilizadas en el programa. Aquí se declaran los atributos y métodos de cada clase, así como la relación de herencia entre ellas. Este archivo permite organizar la estructura del programa de manera clara.

Empleado.cpp

```
#include "Empleado.h"

Empleado::Empleado(string n,int e,double s){
    nombre=n; edad=e; salario=s;
}
double Empleado::calcular(){ return salario; }

void Empleado::mostrar(){
    cout<<nombre<<" Edad:"<<edad<<" Salario:"<<calcular()<<endl;
}


TiempoCompleto::TiempoCompleto(string n,int e,double s,double b)
: Empleado(n,e,s){ bono=b; }

double TiempoCompleto::calcular(){
    return salario+bono;
}


PorHoras::PorHoras(string n,int e,int h,double p)
: Empleado(n,e,0){ horas=h; pago=p; }

double PorHoras::calcular(){
    return horas*pago;
}


Prestaciones::Prestaciones(string s){ seguro=s; }

void Prestaciones::mostrarPrest(){
    cout<<"Seguro:"<<seguro<<endl;
}

CompletoPrest::CompletoPrest(string n,int e,double s,double b,string seg)
: TiempoCompleto(n,e,s,b), Prestaciones(seg){}
```

En esta imagen se observa el archivo Empleado.cpp, donde se implementan los métodos de las clases previamente declaradas. Aquí se define el funcionamiento de cada método, como el cálculo del salario y la forma en que se muestra la información de los empleados.

main.cpp

main.cpp > main()

```cpp
#include "Empleado.h"

int main(){

    Empleado e("Luis",20,5000);
    TiempoCompleto t("Ana",30,8000,2000);
    PorHoras h("Carlos",25,40,100);
    CompletoPrest c("Maria",28,9000,1500,"IMSS");

    e.mostrar();
    t.mostrar();
    h.mostrar();

    cout<<"--- Con prestaciones ---"<<endl;
    c.mostrar();

    return 0;
}
```

En esta imagen se presenta el archivo principal main.cpp, donde se crean los objetos de cada clase y se ejecuta el programa. En esta parte se llaman los métodos para mostrar la información y calcular el salario, lo que permite comprobar que todo funciona correctamente.

EVIDENCIA EJECUCION

Compilación:

Nombre	Estado	Fecha de modificación	Tipo	Tamaño
Empleado.cpp	✓	22/04/2026 04:40 p. m.	Archivo de origen ...	1 KB
Empleado.h	✓	22/04/2026 04:39 p. m.	DevCpp.h	1 KB
main.cpp	✓	22/04/2026 04:41 p. m.	Archivo de origen ...	1 KB
programa.exe	✓	22/04/2026 09:19 p. m.	Aplicación	56 KB

```
MINGW64:/c/Users/LUIS/One

LUIS@COMPUTADORAluis MINGW64 ~/OneDrive/Documentos/Luisale29/luis12/001_Pract
$ g++ main.cpp Empleado.cpp -o programa

LUIS@COMPUTADORAluis MINGW64 ~/OneDrive/Documentos/Luisale29/luis12/001_Pract
$
```

En esta imagen se muestra el proceso de compilación del programa utilizando el compilador g++. Aquí se puede observar que el código no presenta errores, lo que indica que la sintaxis es correcta y que los archivos fueron integrados adecuadamente.

Ejecución:

Empleado.cpp	✓	22/04/2026 04:40 p. m.	Archivo de origen ...	1 KB
Empleado.h	✓	22/04/2026 04:39 p. m.	DevCpp.h	1 KB
main.cpp	✓	22/04/2026 04:41 p. m.	Archivo de origen ...	1 KB
programa.exe	✓	22/04/2026 09:19 p. m.	Aplicación	56 KB

```
MINGW64:/c/Users/LUIS/One

LUIS@COMPUTADORAluis MINGW64 ~/OneDrive/Documentos/Luisale29/luis12/001_Prac
$ g++ main.cpp Empleado.cpp -o programa

LUIS@COMPUTADORAluis MINGW64 ~/OneDrive/Documentos/Luisale29/luis12/001_Prac
$ ./programa
```

En esta imagen se observa la ejecución del programa en la consola. Se muestran los datos de los diferentes tipos de empleados junto con el cálculo de su salario, lo que permite verificar que el programa funciona correctamente.

Resultado en consola:

Empleado.cpp	✓	22/04/2026 04:40 p. m.	Archivo de origen ...	1 KB
Empleado.h	✓	22/04/2026 04:39 p. m.	DevCpp.h	1 KB
main.cpp	✓	22/04/2026 04:41 p. m.	Archivo de origen ...	1 KB
programa.exe	✓	22/04/2026 09:19 p. m.	Aplicación	56 KB

```
MINGW64:/c/Users/LUIS/One

LUIS@COMPUTADORAluis MINGW64 ~/OneDrive/Documentos/Luisale29/luis12/001_
$ g++ main.cpp Empleado.cpp -o programa

LUIS@COMPUTADORAluis MINGW64 ~/OneDrive/Documentos/Luisale29/luis12/001_
$ ./programa
Luis Edad:20 Salario:5000
Ana Edad:30 Salario:10000
Carlos Edad:25 Salario:4000
--- Con prestaciones ---
Maria Edad:28 Salario:10500
Seguro:IMSS
```

En esta imagen se muestra tanto el proceso de compilación como la ejecución del programa en la terminal. Primero, se utiliza el comando g++ para compilar los archivos main.cpp y Empleado.cpp, generando el archivo ejecutable. Posteriormente, se ejecuta el programa, donde se pueden observar los datos de los distintos empleados junto con el cálculo de su salario. Esto demuestra que el programa funciona correctamente y que la herencia fue aplicada de manera adecuada.

ANALISIS DEL PROGRAMA

Una de las principales ventajas de usar herencia en este programa es que permite reutilizar código, ya que no es necesario volver a escribir los mismos atributos y métodos en cada clase. Por ejemplo, las clases derivadas pueden utilizar directamente las características de la clase Empleado, lo que hace que el programa sea más organizado y fácil de entender. Además, facilita la creación de nuevos tipos de empleados sin tener que modificar todo el código.

Si no se utilizara la herencia, el programa sería más largo y repetitivo, ya que cada tipo de empleado tendría que declarar nuevamente variables como nombre, edad y salario, así como sus métodos. Esto haría el código más difícil de mantener y aumentaría la posibilidad de cometer errores.

La parte que resultó más difícil fue comprender y aplicar la herencia múltiple, ya que implica que una clase herede de más de una clase al mismo tiempo. También fue un poco complicado entender cómo funcionan los constructores en las clases derivadas y cómo se conectan con la clase base.

CONCLUSION

En esta práctica aprendí a utilizar la herencia dentro de la programación orientada a objetos, así como a organizar mejor un programa mediante el uso de clases. También comprendí cómo una clase base puede servir para crear otras más específicas y cómo es posible modificar métodos para adaptarlos a diferentes situaciones, como en el cálculo del salario de distintos tipos de empleados.

Este conocimiento me sirve en la vida real para desarrollar programas más ordenados, reutilizables y fáciles de mantener. La herencia permite evitar repetir código y facilita la creación de sistemas más grandes sin que se vuelvan difíciles de entender. Además, ayuda a pensar de manera más lógica al momento de estructurar un programa.

Este tipo de programación se puede aplicar en muchos ámbitos, como en sistemas de empresas, aplicaciones, videojuegos o cualquier software que necesite manejar distintos tipos de datos u objetos con características similares. Por ejemplo, en una empresa se pueden manejar diferentes tipos de empleados, clientes o productos utilizando este mismo concepto.

En conclusión, esta práctica me permitió entender mejor cómo aplicar la programación orientada a objetos en situaciones reales y cómo estos conceptos son útiles para desarrollar software de manera más eficiente y organizada.